Asservissement des roues par retour d'état avec action intégrale

Robot mecanum 4 roues – Navigation

Notations

Les notations ci-dessous sont utilisées dans tout le document. Une grandeur en gras est un vecteur de $\mathbb{R}^4$. L'indice $i \in \{FL, FR, BR, BL\}$ désigne une roue, l'indice $j \in \{x, y, \theta, t\}$ désigne un mode, et l'indice c désigne une consigne.

Symbole	Signification	Unité
Espace des roues		
$n_i,\ \boldsymbol{n}$	vitesse mesurée d'une roue	RPM
$n_{c,i},\ \boldsymbol{n}_c$	consigne de vitesse d'une roue (`setrpm`)	RPM
$r_i,\ \boldsymbol{r}$	commande d'une roue exprimée en vitesse	RPM
$u_i,\ \boldsymbol{u}$	PWM envoyée à l'Arduino	$[-255, 255]$
P_i	caractéristique statique $u \mapsto n$ (polynôme)	RPM
Espace des modes		
$\boldsymbol{T}$	matrice de passage roues → modes	–
$z_j,\ \boldsymbol{z} = \boldsymbol{T}\boldsymbol{n}$	vitesse modale	RPM
$z_{c,j},\ \boldsymbol{z}_c = \boldsymbol{T}\boldsymbol{n}_c$	consigne modale	RPM
$\rho_j,\ \boldsymbol{\rho} = \boldsymbol{T}\boldsymbol{r}$	commande modale	RPM
$q_j,\ \boldsymbol{q}$	intégrale de l'erreur modale (= position)	RPM·s
Robot		
$v_x,\ v_y$	vitesses linéaires du robot (repère robot)	m/s
$\theta,\ \dot{\theta}$	cap et vitesse de rotation	rad, rad/s
d	écart latéral à la trajectoire rectiligne	m
v_0	vitesse d'avance nominale	m/s
Réglage		
τ	constante de temps d'une roue	s
$k_{1,j},\ k_{2,j},\ m_j$	gains du mode j	–, s^{-1}, –
$\omega_0,\ \xi$	pulsation propre et amortissement imposés	rad/s, –
$\Delta t,\ h$	période d'échantillonnage, fenêtre de la moyenne encodeurs	s

Chaîne de commande. Le correcteur calcule $\boldsymbol{\rho}$ (modes). On en déduit $\boldsymbol{r}$ (roues, en RPM), puis $\boldsymbol{u}$ (PWM) :

$$\underbrace{(\boldsymbol{z}, \boldsymbol{q})}_{\text{mesures}} \xrightarrow{\text{retour d'état}} \boldsymbol{\rho} \xrightarrow{\boldsymbol{r}=\boldsymbol{T}^{\mathsf{T}}\boldsymbol{\rho}} \boldsymbol{r} \xrightarrow{u_i = P_i^{-1}(r_i)} \boldsymbol{u} \xrightarrow{\text{moteurs}} \boldsymbol{n} \xrightarrow{\boldsymbol{z}=\boldsymbol{T}\boldsymbol{n}} \boldsymbol{z}.$$

1 Schéma bloc

La figure 1 représente la boucle telle qu'elle est implémentée. Les flèches épaisses portent des vecteurs de $\mathbb{R}^4$. Tout est calculé sur la Raspberry Pi (`control.py` et `state_feedback.py`), sauf le bloc « Arduino, moteurs, roues ».

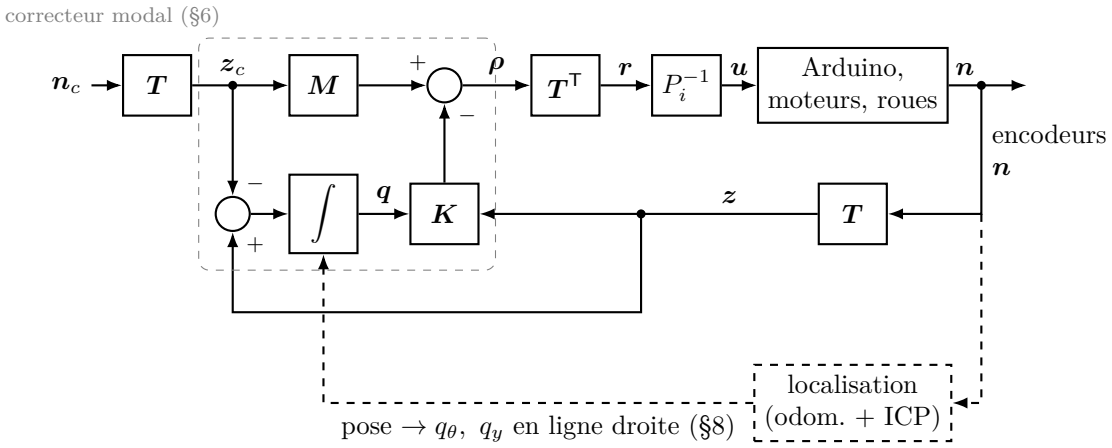


FIGURE 1 – Schéma bloc de l'asservissement implémenté. Le bloc $\boldsymbol{K} = [\boldsymbol{K}_1 \;\; \boldsymbol{K}_2]$ agit sur l'état augmenté $\boldsymbol{x} = (\boldsymbol{z},\ \boldsymbol{q})$: $\boldsymbol{K}\boldsymbol{x} = \boldsymbol{K}_1\boldsymbol{z} + \boldsymbol{K}_2\boldsymbol{q}$, avec $\boldsymbol{K}_1 = \mathrm{diag}(k_{1,j})$, $\boldsymbol{K}_2 = \mathrm{diag}(k_{2,j})$ et $\boldsymbol{M} = \boldsymbol{I}_4 + \boldsymbol{K}_1$. Les deux blocs $\boldsymbol{T}$ passent des roues aux modes ; le bloc $\boldsymbol{T}^\mathsf{T}$ revient des modes aux roues.

On reconnaît la structure du cours (p. 39 et 43) : $\boldsymbol{\rho} = -\boldsymbol{K}\boldsymbol{x} + \boldsymbol{M}\boldsymbol{z}_c = -\boldsymbol{K}_1\boldsymbol{z} - \boldsymbol{K}_2\boldsymbol{q} + \boldsymbol{M}\boldsymbol{z}_c$. La seule différence est que le « système » vu par le correcteur est la chaîne $\boldsymbol{T}^\mathsf{T} \to P_i^{-1} \to$ moteurs $\to \boldsymbol{T}$, qui transforme $\boldsymbol{\rho}$ en $\boldsymbol{z}$.

Correspondance avec le code.

Signal	Variable dans le code	Origine
$\boldsymbol{n}_c$	`wc.target_rpm`	`/robot/command` (`setrpm`, `automatic`)
$\boldsymbol{n}$	`wc.current_rpm`	`/robot/encoders`
$\boldsymbol{z}_c$, $\boldsymbol{z}$	`sf.z_c`, `sf.z`	calculés dans `state_feedback.py`
$\boldsymbol{q}$	`sf.q`	idem (ou pose de `/robot/pos`)
$\boldsymbol{\rho}$, $\boldsymbol{r}$	`sf.rho`, `sf.r`	idem
$\boldsymbol{u}$	`msg.*_wheel_speed`	`/robot/wheels` → Arduino

2 Cinématique

Constantes : $R = 0{,}0325\,\mathrm{m}$ (rayon des roues) et $L = L_X + L_Y = 0{,}20\,\mathrm{m}$.

La cinématique de `localization.py`, avec les roues dans l'ordre FL, FR, BR, BL et les vitesses en RPM, s'écrit :

$$v_x = \tfrac{R}{4}\tfrac{\pi}{30}\,(\;\; n_{FL} + n_{FR} + n_{BR} + n_{BL}), \tag{1}$$

$$v_y = \tfrac{R}{4}\tfrac{\pi}{30}\,(-n_{FL} + n_{FR} - n_{BR} + n_{BL}), \tag{2}$$

$$\dot\theta = \tfrac{R}{4L}\tfrac{\pi}{30}\,(-n_{FL} + n_{FR} + n_{BR} - n_{BL}). \tag{3}$$

3 La matrice $\boldsymbol{T}$

3.1 Définition et rôle

Chaque parenthèse ci-dessus est un « motif » de rotation des roues. On range ces motifs en lignes et on ajoute un quatrième motif, la torsion, qui ne fait pas bouger le robot :

$$\boldsymbol{T} = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{pmatrix} \begin{matrix} \leftarrow \text{mode } x \text{ (avance)} \\ \leftarrow \text{mode } y \text{ (latéral)} \\ \leftarrow \text{mode } \theta \text{ (rotation)} \\ \leftarrow \text{mode } t \text{ (torsion)} \end{matrix} \qquad \boldsymbol{z} = \boldsymbol{T}\boldsymbol{n}. \tag{4}$$

z_j mesure « combien » du motif j est présent dans les vitesses des roues : c'est la projection de $\boldsymbol{n}$ sur le motif j. On passe ainsi d'une description par roue (FL, FR, BR, BL) à une description par mouvement du robot (avance, latéral, rotation, torsion). La déviation du robot se lit alors directement sur z_θ et z_y.

Liens avec les vitesses physiques :

$$z_x = c_v\, v_x, \qquad z_y = c_v\, v_y, \qquad z_\theta = c_\theta\, \dot{\theta}, \tag{5}$$

avec

$$c_v = \frac{2}{R}\frac{30}{\pi} \approx 587{,}6\,\mathrm{RPM}\cdot\mathrm{s/m}, \qquad c_\theta = \frac{2L}{R}\frac{30}{\pi} \approx 117{,}5\,\mathrm{RPM}\cdot\mathrm{s/rad}. \tag{6}$$

3.2 Pourquoi $\boldsymbol{T}^\mathsf{T}\boldsymbol{T} = \boldsymbol{I}_4$

Le coefficient (i, j) de $\boldsymbol{T}\boldsymbol{T}^\mathsf{T}$ est le produit scalaire de la ligne i par la ligne j.

— Deux lignes différentes sont orthogonales. Par exemple, $x \cdot y = \frac{1}{4}(-1 + 1 - 1 + 1) = 0$ et $\theta \cdot t = \frac{1}{4}(-1 + 1 - 1 + 1) = 0$. Les six paires se vérifient de la même façon.

— Chaque ligne est de norme 1 : $\frac{1}{4}(1 + 1 + 1 + 1) = 1$. C'est le rôle du facteur $\frac{1}{2}$.

Donc $\boldsymbol{T}\boldsymbol{T}^\mathsf{T} = \boldsymbol{I}_4$. Comme $\boldsymbol{T}$ est carrée, $\boldsymbol{T}^\mathsf{T}$ est alors son inverse, et l'on a aussi $\boldsymbol{T}^\mathsf{T}\boldsymbol{T} = \boldsymbol{I}_4$. On dit que $\boldsymbol{T}$ est *orthogonale*.

3.3 Pourquoi c'est utile

1. **Retour aux roues sans calcul d'inverse.** Le correcteur produit $\boldsymbol{\rho}$ dans l'espace des modes, et il faut des commandes par roue :

$$\boldsymbol{r} = \boldsymbol{T}^{-1}\boldsymbol{\rho} = \boldsymbol{T}^\mathsf{T}\boldsymbol{\rho}. \tag{7}$$

2. **Découplage.** Si les quatre roues ont la même dynamique, chaque mode z_j obéit à une équation indépendante des autres modes. On le règle donc séparément (démonstration en section 5). L'inversibilité de $\boldsymbol{T}$ suffirait pour ce point. L'orthogonalité garantit en plus les deux points suivants.
3. **Même échelle pour tous les modes.** $\|\boldsymbol{z}\| = \|\boldsymbol{n}\|$ et $\|\boldsymbol{\rho}\| = \|\boldsymbol{r}\|$. Une correction de 10 RPM sur un mode se traduit par ± 5 RPM sur chaque roue, quel que soit le mode. Les gains et les poids LQ des différents modes sont donc comparables. De même, un bruit de mesure de variance σ^2 sur chaque roue (indépendant d'une roue à l'autre) donne un bruit de même variance σ^2 sur chaque mode : $\boldsymbol{T}(\sigma^2\boldsymbol{I})\boldsymbol{T}^\mathsf{T} = \sigma^2\boldsymbol{I}$.
4. **Intérêt de régler les modes séparément.** Avec les mêmes gains k sur tous les modes, on obtient $\boldsymbol{T}^\mathsf{T}(k\boldsymbol{I})\boldsymbol{T} = k\boldsymbol{I}$: la loi est strictement identique à quatre correcteurs indépendants, un par roue. Le gain apporté par l'approche d'état vient donc du fait de donner des gains *différents* aux modes y et θ, qui portent la déviation.

4 Caractéristique PWM → RPM et linéarisation

`wheelBenchmark.cpp` augmente la PWM de 40 à 254 par pas de 2. À chaque pas, il attend 0,5 s puis mesure la vitesse pendant 1 s. Une régression donne, pour chaque roue, un polynôme $n_i = P_i(u_i)$ (`control.py`), par exemple :

$$P_{FL}(u) = 2{,}543 \cdot 10^{-5}u^3 - 0{,}01625\,u^2 + 3{,}696\,u - 136{,}4. \tag{8}$$

Le gain local $G_i = \mathrm{d}P_i/\mathrm{d}u$ varie fortement d'un point de fonctionnement à l'autre :

Roue	Zone morte (PWM)	$P_i(255)$ (RPM)	à 100 RPM PWM	G_i	à 150 RPM PWM	G_i
FL	< 45	171	103	1,15	178	0,33
FR	< 63	167	128	0,96	218	0,39
BR	< 34	171	121	0,99	208	0,30
BL	< 43	174	103	1,13	176	0,37

Linéarisation. On envoie $u_i = P_i^{-1}(r_i)$, calculé par dichotomie (`binary_search`). En régime permanent, on obtient alors $n_i = P_i(P_i^{-1}(r_i)) = r_i$. Le gain statique de r_i vers n_i vaut 1 pour toutes les roues et à toutes les vitesses. C'est ce qui rend les roues « identiques » (section 5).

5 Modèle des roues et dynamique des modes

5.1 Modèle d'une roue (hypothèse)

La dynamique d'une roue n'est pas connue : on la *suppose* du premier ordre. C'est le modèle classique d'un motoréducteur à courant continu, dont la constante de temps électrique (quelques ms) est négligeable devant la constante de temps mécanique. Grâce à la linéarisation de la section 4, le gain statique vaut 1 :

$$\tau\,\dot{n}_i(t) = -n_i(t) + r_i(t), \qquad i \in \{FL, FR, BR, BL\}. \tag{9}$$

En régime permanent ($\dot{n}_i = 0$), on retrouve bien $n_i = r_i$. La valeur $\tau \approx 0{,}1\,\mathrm{s}$ est provisoire : elle doit être mesurée (section 12).

Hypothèses du modèle :

1. les quatre roues ont la même constante de temps τ ;
2. leur gain statique vaut 1 (polynômes P_i à jour) ;
3. chaque roue ne dépend que de sa propre commande : on néglige l'influence du châssis et du sol, qui relient les roues entre elles.

5.2 Écriture vectorielle

On empile les quatre équations :

$$\tau\,\dot{\boldsymbol{n}} = -\boldsymbol{n} + \boldsymbol{r}. \tag{10}$$

5.3 Dynamique des modes

La dynamique des modes n'est pas mesurée : elle se *déduit* du modèle des roues. On multiplie l'équation vectorielle à gauche par $\boldsymbol{T}$:

$$\tau\,\boldsymbol{T}\dot{\boldsymbol{n}} = -\boldsymbol{T}\boldsymbol{n} + \boldsymbol{T}\boldsymbol{r}. \tag{11}$$

$\boldsymbol{T}$ est constante, donc $\boldsymbol{T}\dot{\boldsymbol{n}} = \frac{\mathrm{d}}{\mathrm{d}t}(\boldsymbol{T}\boldsymbol{n}) = \dot{\boldsymbol{z}}$. Par définition, $\boldsymbol{T}\boldsymbol{n} = \boldsymbol{z}$ et $\boldsymbol{T}\boldsymbol{r} = \boldsymbol{\rho}$. On obtient :

$$\boxed{\tau\,\dot{\boldsymbol{z}} = -\boldsymbol{z} + \boldsymbol{\rho}} \qquad \Longleftrightarrow \qquad \tau\,\dot{z}_j = -z_j + \rho_j \quad \text{pour chaque mode } j \in \{x, y, \theta, t\}. \tag{12}$$

Chaque mode est un premier ordre, de même τ que les roues, et ne dépend que de sa propre commande ρ_j.

Exemple (mode rotation). La ligne θ de $\boldsymbol{T}$ donne $z_\theta = \frac{1}{2}(-n_{FL} + n_{FR} + n_{BR} - n_{BL})$. Alors :

$$\begin{aligned}\tau\dot{z}_\theta &= \tfrac{1}{2}\big(-\tau\dot{n}_{FL} + \tau\dot{n}_{FR} + \tau\dot{n}_{BR} - \tau\dot{n}_{BL}\big)\\ &= \tfrac{1}{2}\big(-(r_{FL} - n_{FL}) + (r_{FR} - n_{FR}) + (r_{BR} - n_{BR}) - (r_{BL} - n_{BL})\big) = \rho_\theta - z_\theta.\end{aligned}$$

5.4 Si les roues ne sont pas identiques

Si chaque roue a sa propre constante τ_i, on pose $\boldsymbol{D} = \mathrm{diag}(1/\tau_i)$. Alors :

$$\dot{\boldsymbol{z}} = -(\boldsymbol{T}\boldsymbol{D}\boldsymbol{T}^{\mathsf{T}})\boldsymbol{z} + (\boldsymbol{T}\boldsymbol{D}\boldsymbol{T}^{\mathsf{T}})\boldsymbol{\rho}. \tag{13}$$

$\boldsymbol{T}\boldsymbol{D}\boldsymbol{T}^{\mathsf{T}}$ n'est diagonale que si tous les τ_i sont égaux. Sinon, les modes sont légèrement couplés, d'autant plus que les τ_i diffèrent. L'action intégrale corrige ce petit écart au modèle. L'identification (section 12) permet de vérifier que les τ_i sont proches.

6 Système augmenté d'un mode (action intégrale)

Pour un mode j de consigne $z_{c,j}$ constante, on ajoute l'état intégral :

$$q_j(t) = \int_0^t (z_j(s) - z_{c,j})\,\mathrm{d}s. \tag{14}$$

q_j est une position. z_j est une vitesse, donc q_j est l'écart entre la position parcourue dans le mode j et la position de consigne. Par exemple, pour la rotation : $q_\theta = c_\theta(\theta - \theta_c)$.

Représentation d'état. L'état augmenté est $\boldsymbol{x}_j = (z_j,\ q_j)^{\mathsf{T}}$:

$$\dot{\boldsymbol{x}}_j = \underbrace{\begin{pmatrix} -1/\tau & 0 \\ 1 & 0 \end{pmatrix}}_{\boldsymbol{A}_a} \boldsymbol{x}_j + \underbrace{\begin{pmatrix} 1/\tau \\ 0 \end{pmatrix}}_{\boldsymbol{B}_a} \rho_j + \begin{pmatrix} 0 \\ -1 \end{pmatrix} z_{c,j}. \tag{15}$$

La première ligne est une accélération, la seconde l'écart de vitesse. La vitesse n'apparaît donc qu'une fois dans $\dot{\boldsymbol{x}}_j$.

Commande (cours p. 39 et 43). On applique un retour d'état et une anticipation de consigne :

$$\rho_j = -\boldsymbol{K}_j\boldsymbol{x}_j + m_j\, z_{c,j} = -k_{1,j}\, z_j - k_{2,j}\, q_j + m_j\, z_{c,j}, \qquad m_j = 1 + k_{1,j}. \tag{16}$$

Avec ce choix de m_j, on peut aussi écrire $\rho_j = z_{c,j} - k_{1,j}(z_j - z_{c,j}) - k_{2,j}q_j$.

Schéma bloc d'un mode. D'après la section 5, la chaîne $\boldsymbol{T}^{\mathsf{T}} \to P_i^{-1} \to$ moteurs $\to \boldsymbol{T}$ de la figure 1 équivaut, pour chaque mode, à un premier ordre $1/(1+\tau s)$. La figure 2 montre la boucle obtenue, sur laquelle on place les pôles.

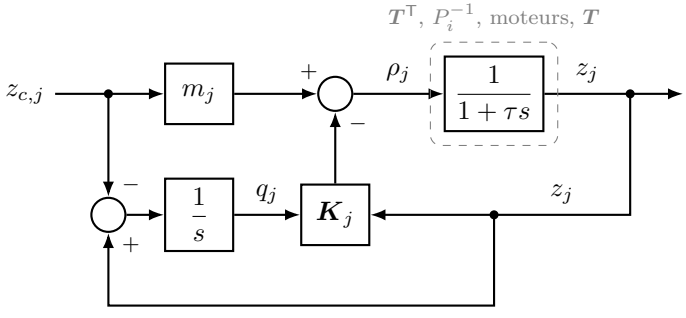


FIGURE 2 – Boucle équivalente pour un mode $j \in \{x, y, \theta, t\}$ (une boucle par mode, toutes indépendantes). Le bloc $\boldsymbol{K}_j$ reçoit l'état augmenté $\boldsymbol{x}_j = (z_j,\ q_j)$ et renvoie $\boldsymbol{K}_j\boldsymbol{x}_j = k_{1,j}z_j + k_{2,j}q_j$.

Conditions d'existence (cours p. 41).

$$\mathrm{rang}\,[\boldsymbol{B}_a \quad \boldsymbol{A}_a\boldsymbol{B}_a] = \mathrm{rang}\begin{pmatrix} 1/\tau & -1/\tau^2 \\ 0 & 1/\tau \end{pmatrix} = 2, \qquad \det\begin{pmatrix} -1/\tau & 1/\tau \\ 1 & 0 \end{pmatrix} = -\frac{1}{\tau} \neq 0. \tag{17}$$

Boucle fermée.

$$\boldsymbol{A}_a - \boldsymbol{B}_a\boldsymbol{K}_j = \begin{pmatrix} -\dfrac{1+k_{1,j}}{\tau} & -\dfrac{k_{2,j}}{\tau} \\ 1 & 0 \end{pmatrix}, \qquad \chi_j(s) = s^2 + \frac{1+k_{1,j}}{\tau}\, s + \frac{k_{2,j}}{\tau}. \tag{18}$$

Régime permanent. On pose $\dot{q}_j = 0$, ce qui donne $z_j = z_{c,j}$: il n'y a pas d'erreur statique, même avec une perturbation constante. Avec $\dot{z}_j = 0$, on trouve ensuite $q_j = 0$ en l'absence de perturbation.

7 Placement de pôles

On impose $\chi_j(s) = s^2 + 2\xi\omega_0 s + \omega_0^2$, c'est-à-dire des pôles en $\lambda_{1,2} = -\xi\omega_0 \pm j\,\omega_0\sqrt{1-\xi^2}$. On en déduit :

$$\boxed{k_{1,j} = 2\xi\omega_0\tau - 1, \qquad k_{2,j} = \omega_0^2\,\tau, \qquad m_j = 1 + k_{1,j}} \tag{19}$$

Valeurs proposées pour $\tau = 0{,}1\,\mathrm{s}$:

Modes	ω_0 (rad/s)	ξ	Pôles	$\boldsymbol{K}_j = [k_{1,j} \;\; k_{2,j}]$	m_j
x, t	6	0,9	$-5{,}4 \pm 2{,}62j$	[0,08 3,6]	1,08
y, θ	8	0,9	$-7{,}2 \pm 3{,}49j$	[0,44 6,4]	1,44

Les modes y et θ, qui portent la déviation, sont réglés plus raides que l'avance.

Contrainte liée à la mesure. La moyenne glissante des encodeurs, de fenêtre h, introduit un retard d'environ $h/2$. Il faut $\omega_0\, h/2 \lesssim 0{,}4$, d'où $h = 0{,}1\,\mathrm{s}$ publié à 50 Hz. Avec la configuration actuelle ($h = 0{,}3\,\mathrm{s}$, 10 Hz), la simulation donne 52 % de dépassement pour $\omega_0 = 8$.

8 Mesurer q_θ et q_y avec la pose du robot

Pour une consigne en ligne droite ($z_{c,y} = z_{c,\theta} = 0$), avec (x_0, y_0, θ_0) la pose à l'instant de la consigne :

$$q_\theta = c_\theta\,(\theta - \theta_0), \tag{20}$$

$$q_y = c_v\, d, \qquad d = -\sin\theta_0\,(x - x_0) + \cos\theta_0\,(y - y_0). \tag{21}$$

La pose (x, y, θ) est lue sur `/robot/pos`. Si cette pose provient uniquement des encodeurs, on retrouve exactement l'intégrale de la section 6. L'intérêt vient de la correction ICP du lidar, qui voit le glissement des rouleaux.

Pour le mode x, on garde l'intégrale calculée aux encodeurs. Si l'on bouclait sur la position x et que le robot était bloqué, q_x divergerait (emballement de l'intégrateur).

9 Variante : modes y et θ couplés (commande LQ)

À vitesse d'avance v_0, une erreur de cap crée une dérive latérale :

$$\dot{d} = v_x \sin(\theta - \theta_0) + v_y \cos(\theta - \theta_0) \approx v_0(\theta - \theta_0) + v_y \quad \Longrightarrow \quad \dot{q}_y = z_y + \frac{v_0}{L}\, q_\theta, \tag{22}$$

car $c_v/c_\theta = 1/L$. On regroupe les modes y et θ dans un seul système d'état :

$$\boldsymbol{x} = \begin{pmatrix} z_y \\ z_\theta \\ q_\theta \\ q_y \end{pmatrix}, \quad \dot{\boldsymbol{x}} = \underbrace{\begin{pmatrix} -1/\tau & 0 & 0 & 0 \\ 0 & -1/\tau & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & v_0/L & 0 \end{pmatrix}}_{\boldsymbol{A}} \boldsymbol{x} + \underbrace{\frac{1}{\tau}\begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}}_{\boldsymbol{B}} \begin{pmatrix} \rho_y \\ \rho_\theta \end{pmatrix}, \quad \begin{pmatrix} \rho_y \\ \rho_\theta \end{pmatrix} = -\boldsymbol{K}\boldsymbol{x}. \tag{23}$$

Le système est commandable (rang 4). On minimise $J = \int_0^\infty (\boldsymbol{x}^\mathsf{T}\boldsymbol{Q}\boldsymbol{x} + \boldsymbol{\rho}^\mathsf{T}\boldsymbol{R}\boldsymbol{\rho})\,\mathrm{d}t$, avec des poids choisis par normalisation (cours p. 29) :

$$\boldsymbol{Q} = \mathrm{diag}\Big(\tfrac{1}{50^2},\ \tfrac{1}{50^2},\ \tfrac{1}{(c_\theta \cdot 2°)^2},\ \tfrac{1}{(c_v \cdot 0{,}02\,\mathrm{m})^2}\Big), \qquad \boldsymbol{R} = \tfrac{1}{20^2}\boldsymbol{I}_2. \tag{24}$$

Pour $v_0 = 0{,}34\,\mathrm{m/s}$ (100 RPM), on obtient :

$$\boldsymbol{K} = \begin{pmatrix} 0{,}22 & 0{,}04 & 0{,}67 & 1{,}64 \\ 0{,}04 & 0{,}47 & 5{,}09 & 0{,}46 \end{pmatrix}, \qquad \mathrm{Sp}(\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}) = \{-1{,}72;\ -5{,}10;\ -9{,}46;\ -10{,}65\}. \tag{25}$$

$\boldsymbol{K}$ dépend de v_0 : on le recalcule (`lqr`) pour chaque vitesse nominale.

10 Loi de commande complète (période $\Delta t = 20\,\mathrm{ms}$)

On note $\boldsymbol{K}_1 = \mathrm{diag}(k_{1,j})$, $\boldsymbol{K}_2 = \mathrm{diag}(k_{2,j})$ et $\boldsymbol{M} = \boldsymbol{I}_4 + \boldsymbol{K}_1$. À chaque pas k :

$$\boldsymbol{z}_k = \boldsymbol{T}\,\boldsymbol{n}_k, \qquad \boldsymbol{z}_c = \boldsymbol{T}\,\boldsymbol{n}_c, \qquad \text{(passage aux modes)} \tag{26}$$

$$\boldsymbol{q}_{k+1} = \boldsymbol{q}_k + \Delta t\,(\boldsymbol{z}_k - \boldsymbol{z}_c), \qquad \text{(sauf } q_\theta, q_y : \S 8) \tag{27}$$

$$\boldsymbol{\rho}_k = -\boldsymbol{K}_1\boldsymbol{z}_k - \boldsymbol{K}_2\boldsymbol{q}_k + \boldsymbol{M}\boldsymbol{z}_c, \qquad \text{(retour d'état)} \tag{28}$$

$$\boldsymbol{r}_k = \boldsymbol{T}^\mathsf{T}\boldsymbol{\rho}_k, \qquad \text{(retour aux roues)} \tag{29}$$

$$u_{i,k} = \mathrm{sat}_{[-255,\,255]}\Big(\mathrm{sign}(r_{i,k})\,P_i^{-1}\big(|r_{i,k}|\big)\Big). \qquad \text{(linéarisation)} \tag{30}$$

Anti-emballement (anti-windup). Si une roue sature, on n'intègre pas $\boldsymbol{q}$ à ce pas (intégration conditionnelle).

11 Exemple : que vaut $\boldsymbol{\rho}$ après `setrpm 100` ?

$\boldsymbol{\rho}$ est une grandeur interne. L'utilisateur ne donne jamais de consigne (x, y, θ) : il fixe seulement $\boldsymbol{n}_c$ (`setrpm`). La consigne modale $\boldsymbol{z}_c = \boldsymbol{T}\boldsymbol{n}_c$ n'est qu'une réécriture de $\boldsymbol{n}_c$, calculée dans `control.py`. De même, $\boldsymbol{\rho}$ n'est jamais envoyé : il est immédiatement converti en $\boldsymbol{r} = \boldsymbol{T}^\mathsf{T}\boldsymbol{\rho}$, puis en PWM.

Sans erreur, la commande est la consigne. Si $\boldsymbol{z} = \boldsymbol{z}_c$ et $\boldsymbol{q} = \boldsymbol{0}$, alors $\boldsymbol{\rho} = \boldsymbol{z}_c$ et $\boldsymbol{r} = \boldsymbol{T}^\mathsf{T}\boldsymbol{T}\,\boldsymbol{n}_c = \boldsymbol{n}_c$ (ici sert $\boldsymbol{T}^\mathsf{T}\boldsymbol{T} = \boldsymbol{I}_4$). On envoie alors $u_i = P_i^{-1}(n_{c,i})$: c'est l'anticipation seule. Dans tous les cas, on peut écrire

$$\boldsymbol{\rho} = \boldsymbol{z}_c + \underbrace{\big(-\boldsymbol{K}_1(\boldsymbol{z} - \boldsymbol{z}_c) - \boldsymbol{K}_2\boldsymbol{q}\big)}_{\text{correction modale}}, \qquad \boldsymbol{r} = \boldsymbol{n}_c + \boldsymbol{T}^\mathsf{T}\big(-\boldsymbol{K}_1(\boldsymbol{z} - \boldsymbol{z}_c) - \boldsymbol{K}_2\boldsymbol{q}\big). \tag{31}$$

Application numérique. On prend $\boldsymbol{n}_c = (100, 100, 100, 100)$, d'où $\boldsymbol{z}_c = \boldsymbol{T}\boldsymbol{n}_c = (200,\ 0,\ 0,\ 0)$: de l'avance, sans latéral ni rotation. La consigne « aller droit » est donc implicite : $z_{c,y} = z_{c,\theta} = 0$.

Supposons que le robot dévie vers la gauche, avec $\boldsymbol{n} = (96,\ 104,\ 102,\ 98)$ et $\boldsymbol{q} = (0;\ 0{,}5;\ 2;\ 0)$. La valeur $q_\theta = 2$ correspond à un écart de cap d'environ 1°.

	x	y	θ	t
$\boldsymbol{z} = \boldsymbol{T}\boldsymbol{n}$	200	2	6	0
$\boldsymbol{\rho}$	200	$-0{,}44 \cdot 2 - 6{,}4 \cdot 0{,}5 = -4{,}08$	$-0{,}44 \cdot 6 - 6{,}4 \cdot 2 = -15{,}44$	0

	FL	FR	BR	BL
$\boldsymbol{r} = \boldsymbol{T}^\mathsf{T}\boldsymbol{\rho}$ (RPM)	109,8	90,2	94,3	105,7
PWM sans correction, $P_i^{-1}(100)$	103	128	121	103
PWM envoyée, $u_i = P_i^{-1}(r_i)$	112	119	116	108

Les roues gauches sont accélérées et les roues droites ralenties : le robot revient vers la droite.

12 Identification de τ

Principe. On mesure la réponse des moteurs *seuls*, en boucle ouverte : les PWM sont envoyées directement, sans le correcteur. Avec `setrpm`, on mesurerait la boucle fermée, c'est-à-dire les pôles que l'on a soi-même placés, et non le moteur. Pour chaque roue, la PWM est calculée avec P_i^{-1}, de sorte que les quatre roues visent la même vitesse.

Commande. Robot sur cales, nœuds `control`, `serial` et `encoders` lancés :

```
ros2 run robot wheel_step
```

L'essai applique trois fois un échelon r : 60 RPM $\to$ 120 RPM, soit $u_i = P_i^{-1}(60) \to P_i^{-1}(120)$. L'échelon part d'une vitesse non nulle pour rester hors de la zone morte, où le frottement sec fausserait la mesure.

Modèle ajusté. La réponse mesurée est celle d'un premier ordre, retardée d'un retard pur d (passage par `control`, le port série et l'Arduino), puis vue à travers la moyenne glissante de largeur h des encodeurs. Avec $s = t - d$, la réponse normalisée vaut :

$$y(t) = \begin{cases} \dfrac{1}{h}\Big(s - \tau\big(1 - e^{-s/\tau}\big)\Big) & 0 \le s < h, \\ 1 - \dfrac{\tau}{h}\Big(e^{-(s-h)/\tau} - e^{-s/\tau}\Big) & s \ge h. \end{cases} \tag{32}$$

On ajuste τ et d par moindres carrés. La lecture manuelle $\tau \approx t_{63\,\%} - h/2$ surestime τ de la valeur du retard d.

Résultat. L'outil affiche τ et d pour chaque roue, ainsi que la vitesse atteinte. Si celle-ci s'écarte de plus de 5 % de 120 RPM, les polynômes P_i ne sont plus à jour : il faut les remesurer d'abord (`wheel_benchmark`). On reporte la moyenne des τ dans `WHEEL_TAU` ; les gains $k_{1,j}$, $k_{2,j}$ et m_j sont alors recalculés automatiquement avec les formules de la section 7.